

Smart Lender

Loan Eligibility Prediction System

Abstract

Smart Lender is an end-to-end Machine Learning web application designed to automate and streamline the process of assessing loan eligibility. By leveraging historical banking and credit application data, the system predicts whether an applicant qualifies for a loan. The underlying intelligence is powered by an optimized machine learning classifier trained on various key applicant characteristics. To make the model accessible and actionable for everyday users and credit officers, it is wrapped in an interactive, responsive Flask web application featuring a modern glass morphism design, real-time risk indicators, local draft saving, and live scenario presets.

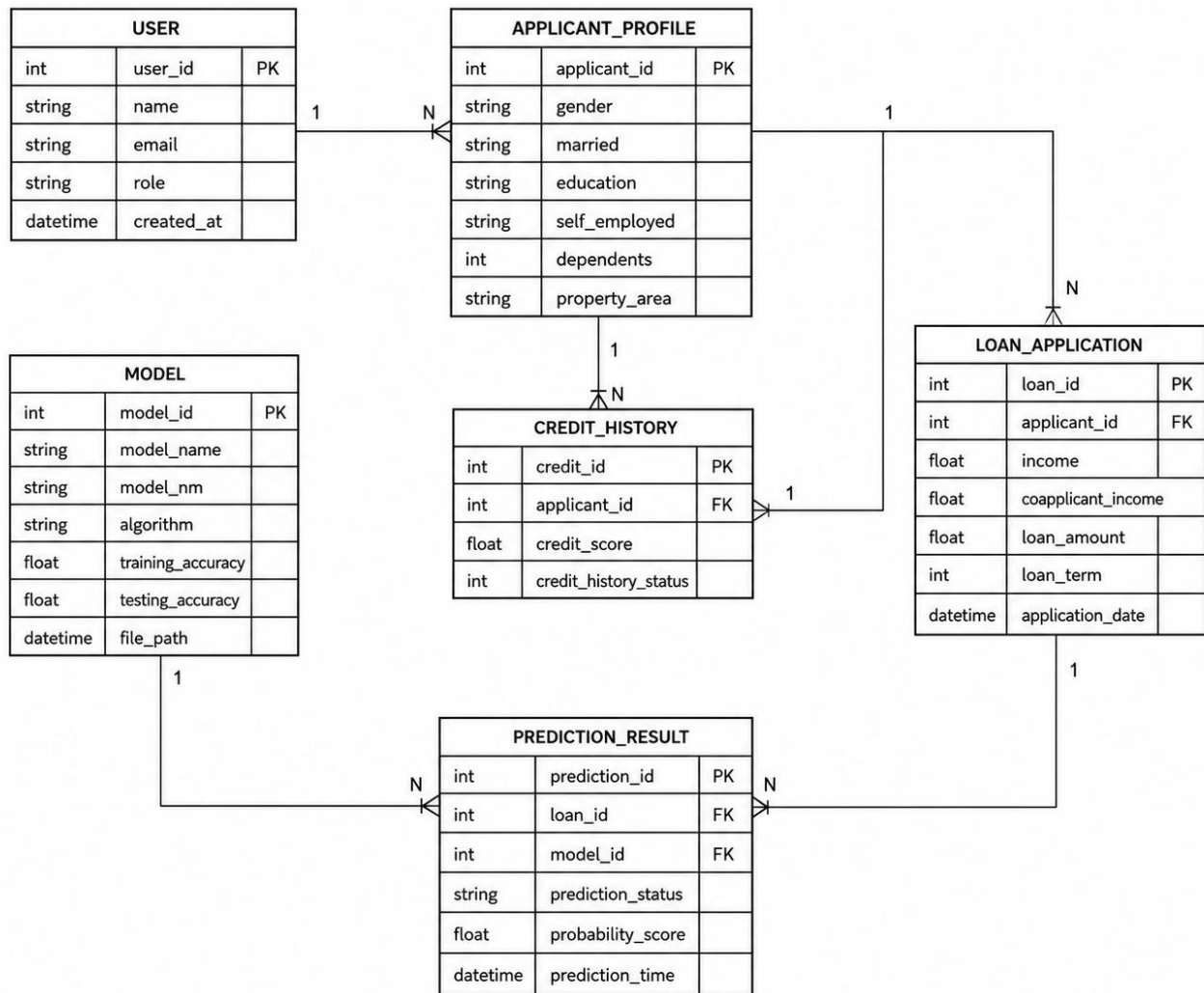
Problem Statement

In traditional banking institutions, loan evaluation is a highly manual, labor-intensive, and time-consuming process. Credit analysts must sift through numerous application forms to assess variables such as personal demographics, income levels, requested loan amounts, debt terms, and credit histories. This manual workflow is:

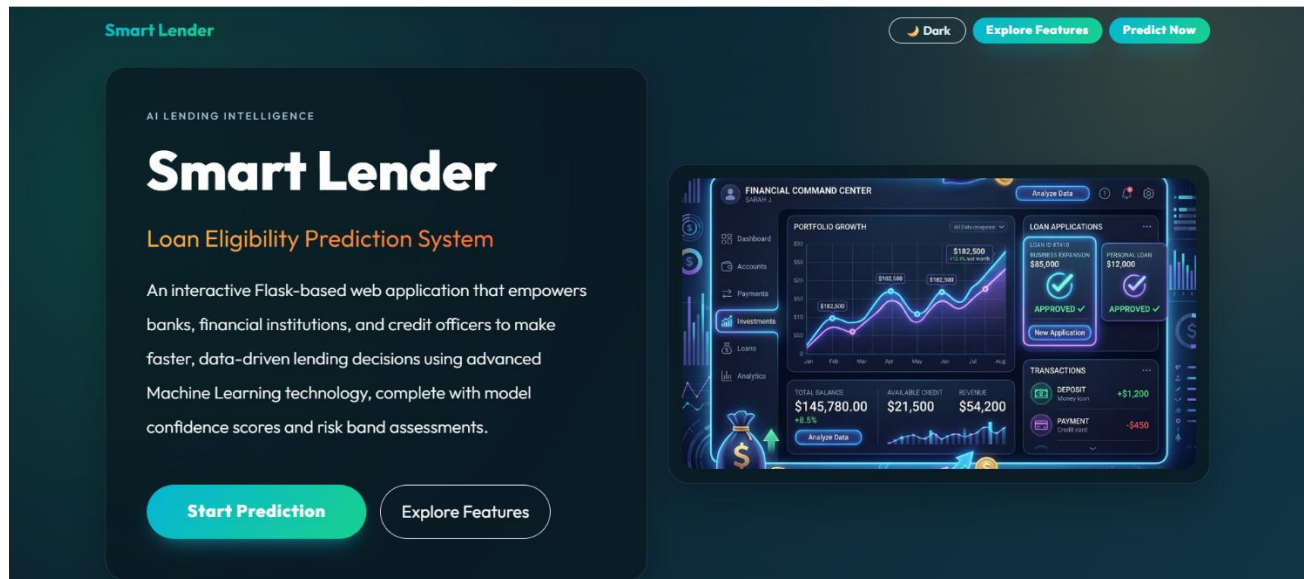
1. **Inefficient:** Processing a single application can take several days or weeks, delaying financial transactions.
2. **Prone to Human Error:** Complex interactions between multiple indicators (e.g., debt-to-income ratio vs. credit history) can be easily misjudged or inconsistently applied.
3. **Inflexible:** Traditional criteria are often rigid and do not provide granular risk assessments or confidence metrics.

Goal: Establish an intelligent, automated, and real-time eligibility assessment system that evaluates applicant profiles instantly, estimates financial burden, and provides classification confidence scores to facilitate fast, accurate, and consistent loan decision-making.

Entity Relationship Diagram:

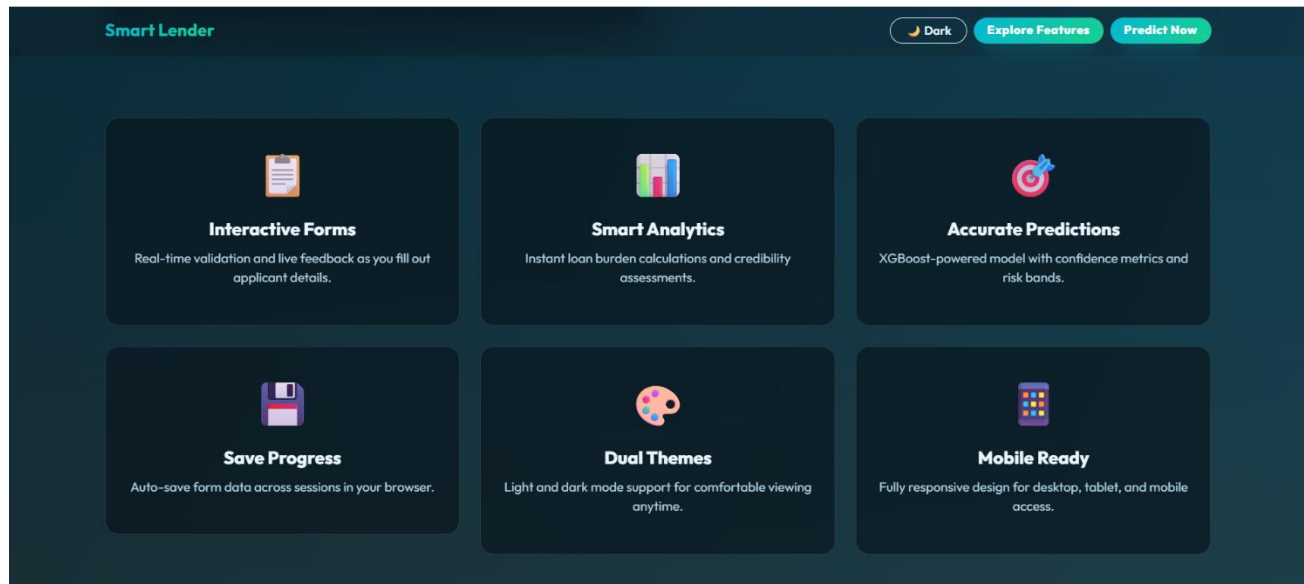


SmartLender is a complete, end-to-end Machine Learning web application designed to evaluate loan applicants and predict their loan eligibility based on key demographic, financial, and credit history variables. By deploying an optimized machine learning classifier integrated with a responsive, modern banking-themed web interface, SmartLender streamlines the traditional, manual credit review process.



The web-based platform allows users to input applicant characteristics and immediately receive eligibility predictions. Feature details include:

- **Predictive Inference Engine:** Computes eligibility outcomes using an optimized XGBoost classifier trained on a structured historical loan application dataset.
- **Interactive Banking UI:** Implements a modern glassmorphism design with responsive widgets and dynamic light/dark theme toggles.
- **Form Progress Tracker:** A real-time visual progress bar indicating input completion percentage.
- **Quick Scenario Presets:** Pre-defined buttons allowing evaluators to load representative low-risk or high-risk test cases instantly.
- **Live Loan Burden Indicator:** Dynamically computes and displays the ratio of requested loan amount to the total annual household income.
- **Profile Hint Engine:** Evaluates inputs programmatically to output real-time profile caution alerts or positive strength signals.
- **Browser Storage Integration:** Enables saving and restoring form input drafts locally using browser localStorage.
- **Result Dashboard:** Displays the prediction status ("Approved" or "Rejected") accompanied by a confidence percentage and risk assessment band.



Features

1. Machine Learning Core

Optimized XGBoost Classifier: Selected over multiple models (Decision Trees, K-Nearest Neighbors, Random Forest) due to its high accuracy (~82%).

Robust Feature Engineering: Utilizes key predictors including:

Demographic features: Gender, Married, Dependents, Education, Self-Employed.

Financial features: Applicant Income, Coapplicant Income, Loan Amount, Loan Amount Term.

Credit metrics: Credit History, Property Area.

Robust Preprocessing: Automatically standardizes input variables using a fitted 'StandardScaler' for reliable model ingestion.

2. Modern & Interactive Frontend

Glassmorphism Theme UI: A responsive, banking-themed aesthetic with a dual-theme toggle (Dark and Light modes).

Interactive Form Completion Progress Bar: Real-time feedback displaying the percentage of required inputs filled.

Estimated Loan Burden Calculator: Dynamically computes and displays the annual debt burden

Live Profile Hint Engine: Generates instant alerts highlighting profile strengths (e.g., good credit record with low risk) or weaknesses (e.g., poor credit history with high loan burden) as fields are modified.

Quick Preset Scenarios: Quick-load buttons to populate the form instantly with typical profiles (Low Risk, High Risk, or Clear form) for easy testing.

Browser-Based Draft Save & Restore: Enables users to save their input progress locally ('localStorage') and restore it with a single click.

3. Detailed Results Dashboard

Instant Verdict: Displays clear “Approved” or “Rejected” decisions.

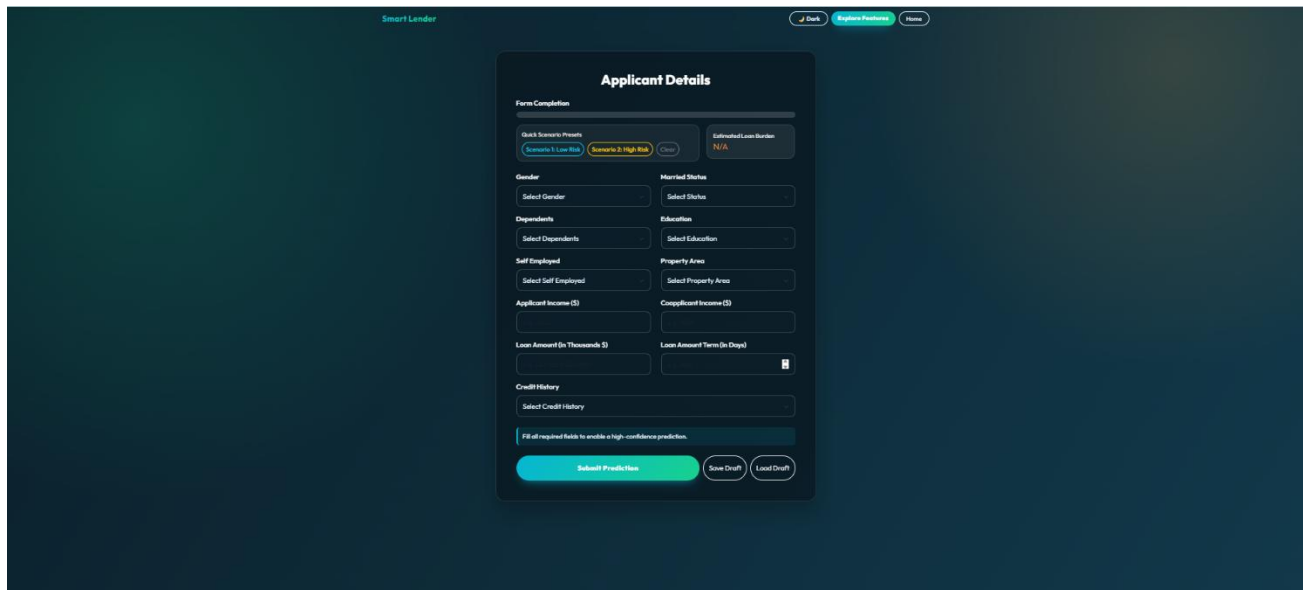
Risk Band Classification: Categorizes applicants into distinct risk levels using probability outputs:

Low Risk: Confidence $\geq 75\%$

Moderate Risk: Confidence 50%-74.99%

High Risk: Confidence $< 50\%$

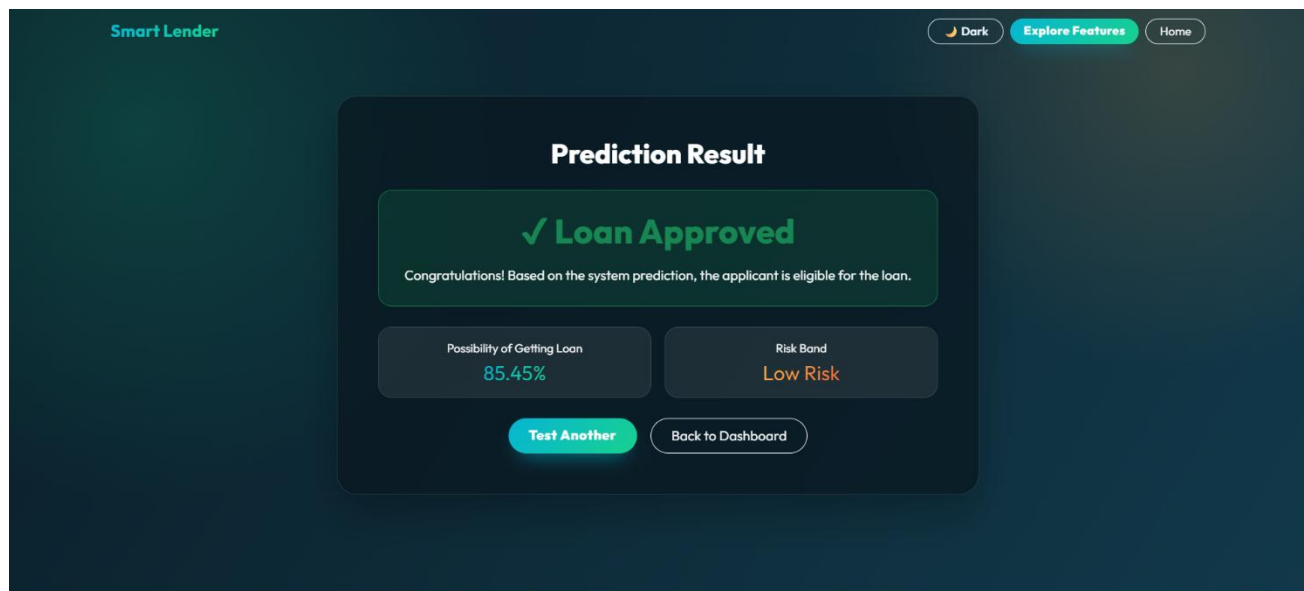
Dynamic Color Themes: Adapts interface colors contextually based on prediction status (green for approvals, red for rejections, orange for errors).



Tech Stack

Layer	Component / Technology	Purpose
Frontend UI	HTML5, CSS3 (Vanilla & custom CSS variables)	Document structure and styling
Frontend Framework	Bootstrap v5.3	Responsive grids, navigation, and theme utilities
Client-Side Logic	JavaScript (ES6+)	Form draft saving, real-time burden ratio, hints, progress bar
Back-end Server	Flask v3.1.3 (Python)	Routing, form validation, and model serving
Machine Learning	Scikit-Learn v1.2.0+, XGBoost v1.7.0+	Model pipeline, scaling, and classification
Data Processing	Pandas v1.5.0+, NumPy v1.20.0+	Tabular manipulation and numerical vector operations

Data Imbalance Handling	Imbalanced-learn v0.10.0+	Optimizing dataset balance during model training
Model Serialization	Pickle	Exporting/importing trained model (rdf.pkl) and scale (scale.pkl)
Deployment / Hosting	Render	To deploy the application



Project Workflow

Model Selection and Architecture

- **Activity 1.1 — Data Preparation:** Load the raw loan prediction datasets (loan_prediction.csv and loan_prediction.xlsx) from the /Dataset directory. Perform exploratory data analysis (EDA), drop metadata identifiers (like Loan_ID), impute null fields, and map categorical values to numeric labels.
- **Activity 1.2 — Model Comparison:** Train and evaluate multiple machine learning classifiers using Training/Loan_Prediction_using_ML.ipynb:
 - *Decision Tree Classifier:* ~76% test accuracy
 - *K-Nearest Neighbors:* ~71% test accuracy
 - *Random Forest Classifier:* ~80% test accuracy
 - *XGBoost Classifier:* ~**82% test accuracy**
- **Activity 1.3 — Model Serialization:** Save the best-performing model (XGBoost) as rdf.pkl and the features StandardScaler as scale.pkl directly inside the /Flask web application directory using Python's pickle library.
- **Activity 1.4 — Workspace Layout Configuration:** Establish the Flask server files, routing code (app.py), UI templates, styling assets, and dependencies metadata (requirements.txt, runtime.txt).

Core Functionalities Development

- **Activity 2.1 — Input Parsing and Validation:** Define server-side input rules

for all 11 fields (FIELD_RULES dictionary) in app.py. Check that fields match their types (integer/float), are within range constraints (e.g. non-negative values for incomes/amount), and belong to categorical sets (e.g., Gender in {0, 1}).

- **Activity 2.2 — Prediction Pipeline Integration:** Build logic in the route handler to feed validated features vector into the loaded scaler (scale.pkl), compute inference predictions ([0] or [1]) with the model (rdf.pkl), and retrieve prediction probabilities for confidence assessment.

App.py Development

- **Activity 3.1 — Server Engine Implementation:** Write the main Flask script app.py:
 - GET /: Renders the index landing page (home.html).
 - GET /predict: Renders the multi-feature applicant form UI (predict.html).
 - POST /submit: Processes request forms, handles validation failures by displaying structured error templates, performs standard scaling + model prediction, and redirects outcomes to the result screen (submit.html).
 - get_risk_band(probability) helper: Maps model prediction probabilities to risk tiers

Frontend Development

- **Activity 4.1 — User Interface Design:** Design a dark glassmorphism layout using Bootstrap 5 and custom CSS style tokens (such as frosted-glass visual blurs, dynamic color gradients, and glowing submit buttons). Introduce light/dark stylesheet themes using custom document attributes.
- **Activity 4.2 — Dynamic Script Interactions:** Develop client-side script events in script.js:
 - *Form Completion Meter:* Loops through required form tags and calculates percentage status as fields are modified.
 - *Live Loan Burden Indicator*
 - *Interactive Presets:* Implements preset load vectors to auto-populate high-risk, low-risk, or clear scenarios.
 - *Draft save/load handler:* Hook local storage events to serialize form states in the browser.

Deployment

Activity 5.1 — Render Environment Preparation:

Initialize the local Python environment, install all required dependencies using pip install -r requirements.txt, verify the Flask application by running python Flask/app.py, and ensure the user interface, machine learning model, and prediction functionality operate correctly before deployment.

Activity 5.2 — Render Deployment Configuration:

Configure the application for deployment on the Render cloud platform by connecting the GitHub repository, specifying the appropriate Root Directory, setting the Build Command as pip install -r requirements.txt, and the Start Command as gunicorn app:app (or cd Flask && gunicorn app:app if the application resides in the Flask subfolder). Ensure all required dependencies are listed in requirements.txt.

Activity 5.3 — Render Cloud Deployment:

Deploy the Smart Lender application by creating a Render Web Service linked to the GitHub repository. Render automatically clones the repository, installs dependencies, builds the application, and hosts it on a public URL. Validate the deployed application by testing the user interface, loan eligibility prediction, and machine learning inference in the live cloud environment.

Installation Steps

Prerequisites

Python 3.9, 3.10, or 3.11 installed.
'pip' package manager.

Steps

1. Clone or Download the Project Repository:

Navigate to the project root directory.

2. Install Required Dependencies:

Run the following command to install the required libraries listed in 'requirements.txt':

```
bash
pip install -r requirements.txt
```

3. (Optional) Run Model Training:

To train models and export the serialized files ('rdf.pkl' and 'scale.pkl'), run the Jupyter Notebook:

Training/Loan_Prediction_using_ML.ipynb

4. Start the Flask Development Server:

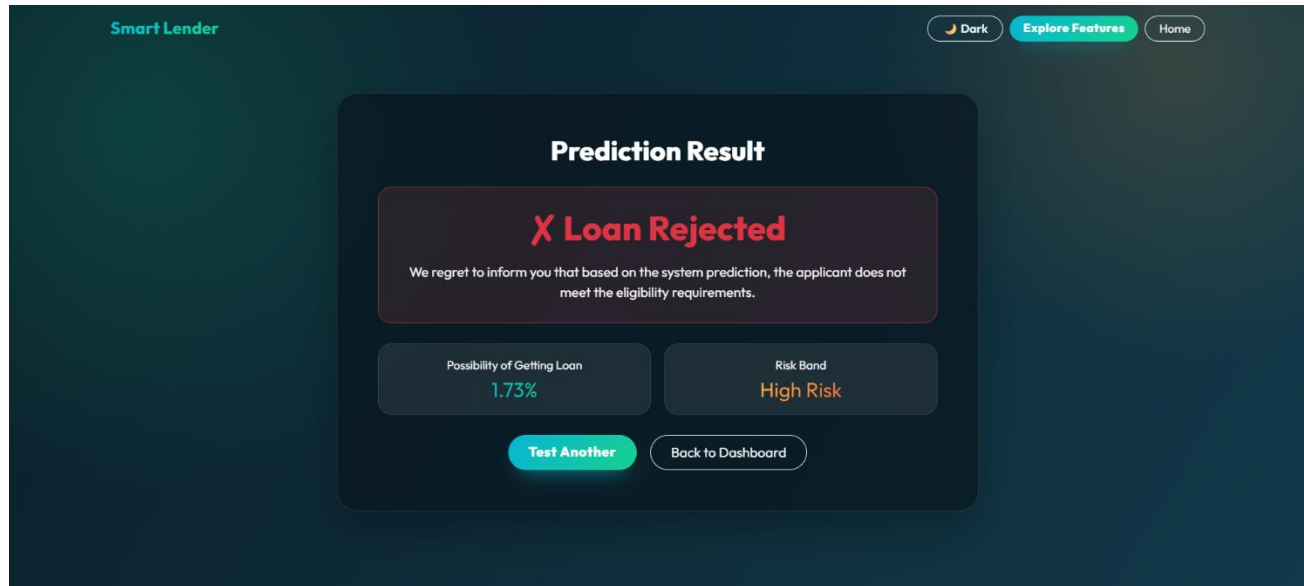
Change to the 'Flask/' directory and execute the server:

```
bash
cd Flask
python app.py
```

5. Access the Web Interface:

Open your browser and navigate to:

```
url
http://127.0.0.1:5000
```

Usage Instructions

1. Accessing the Home Page:

Review the application highlights.

Click the main action button to proceed to the **Loan Eligibility Form**.

2. Filling Out the Application Form:

Manually fill out the demographic and financial details.

Alternatively, click **Scenario 1: Low Risk** or **Scenario 2: High Risk** under **Quick Scenario Presets** to automatically fill the inputs with sample values.

Use the **Save Draft** button to store inputs temporarily in your browser, and **Load Draft** to restore them later.

Review the **Estimated Loan Burden** and **Live Profile Hint** indicators at the bottom to gauge profile parameters.

3. Submitting & Viewing Results:

Click **Submit Prediction**.

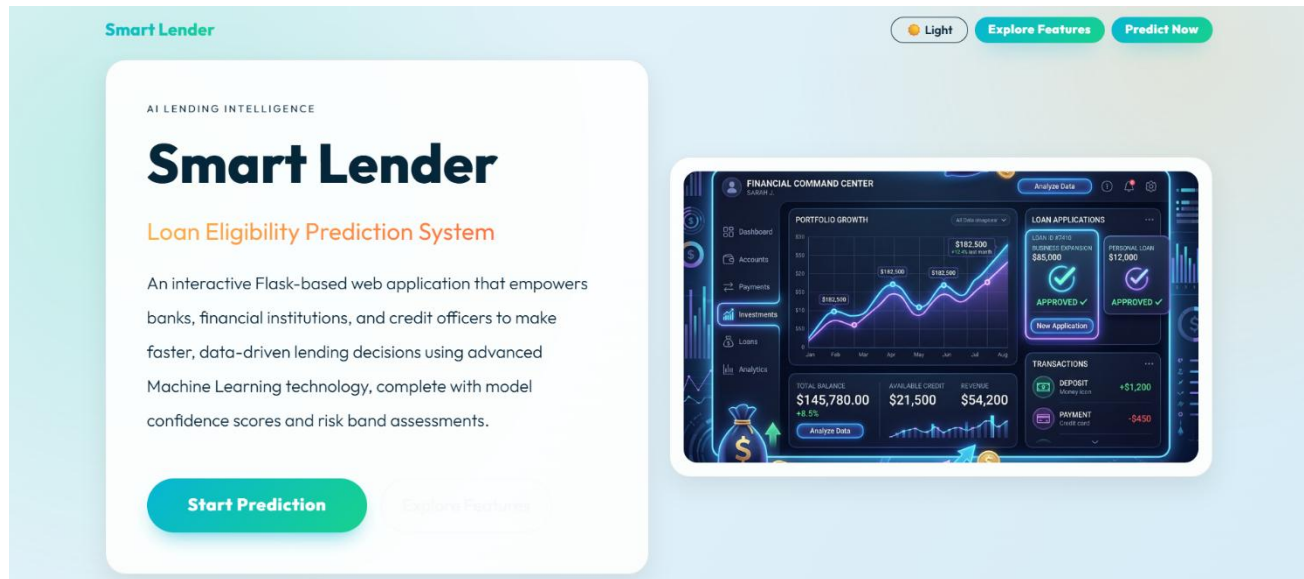
The application will validate your inputs and display:

Decision: Loan Approved (Green) or Loan Rejected (Red).

Confidence Score: Percentage probability computed by the classifier.

Risk Band: Low, Moderate, or High Risk category.

Click "Predict for Another Applicant" to return to the form.



Future Scope

1. Explainable AI (XAI):

Integrate frameworks like SHAP (SHapley Additive exPlanations) or LIME to show visual explanations of which features (e.g., credit history, loan amount) contributed most to the decision, ensuring transparent loan assessments.

2. Automated Third-Party Integration:

Establish API connections with credit bureaus (e.g., Experian, Equifax) to securely pull credit history records in real-time, removing manual input steps and preventing fraud.

3. Multi-Model Consensus & Voting:

Implement a soft-voting ensemble classifier combining XGBoost, Random Forest, and Gradient Boosting algorithms to improve prediction performance and generalize better on outliers.

4. Administrative Controls & Analytics Dashboard:

Introduce user access levels (e.g., Applicant, Credit Officer, Admin) with dashboards tracking approval rates, demographic distributions, and manual override history.

5. Currency Localization:

Support automatic currency conversion and localized formatting based on geographic region, allowing international eligibility verification.

Conclusion

SmartLender showcases how tabular banking datasets can be used to build reliable machine learning microservices. By comparing multiple training methods (Decision Tree, KNN, Random Forest, and XGBoost), the project implements a highly accurate model. The companion Flask application, styled with a modern glassmorphism UI, provides an intuitive interface with real-time indicators like loan burden calculations, interactive test presets, and local draft recovery. Future scaling work could include incorporating credit bureau API connectors, deploying database tables for secure administrative user roles, and integrating explanation graphs (such as SHAP value charts) to supply clear rationales for prediction rejections.

Team Members

Team Lead - Suvvari Ankitha

Member 1 - Sai Geethika Bourouthu

Member 2 - Mohana Swaroop Kukkala